

Detekcja elementów rzutów architektonicznych — dokumentacja techniczna

1. Wstęp

Celem projektu jest automatyczne wykrywanie trzech klas elementów na rzutach architektonicznych: **ścian (wall)**, **drzwi (door)** i **okien (window)**. Model bazuje na architekturze **YOLO11s** (wariant small, ~9.4 M parametrów) w trybie detekcji (detect, nie OBB).

2. Dane treningowe

2.1 Źródła

Dane pozyskano z platformy **Roboflow** z czterech zbiorów:

Zbiór	Link	Klasy	Obrazy (train)
cubiCasa5k-2	roboflow	door/wall/window	4 178
Floor Plan Walls	roboflow	door/wall/window	66
ArchVision Wall Detect v2	roboflow	wall	4 278
Wall Detection v1	roboflow	wall	4 351

2.2 Problem

Żaden z dostępnych zbiorów nie zawierał wszystkich trzech klas z poprawnymi adnotacjami w jednym miejscu. Zbiory **cubiCasa5k-2** i **Floor Plan Walls** miały deklarowane 3 klasy, ale brakowało w nich części adnotacji (głównie drzwi i okien). Zbiory **ArchVision Wall Detect** i **Wall Detection** zawierały wyłącznie ściany.

2.3 Pseudo-labeling

Aby uzyskać kompletny zbiór 3-klasowy, opracowano pipeline pseudo-labelingu:

1. Wytrenowano model pośredni **doors_windows_v2** na zbiorach **~/data/merged_doors_windows/** (3 źródła: Floor Plan Multiple v2, DoorWindow2Door v1, Floor Plans 500 v2 — łącznie 1 999 obrazów, 2 klasy: door=0, window=1). Hiperparametry: yolo11s.pt, imgsiz=1024, batch=16, epochs=30, AdamW, cos_lr.

2. **Predykcja na zbiorach ścian** — model `doors_windows_v2` został użyty do wykrycia drzwi i okien na obrazach ze zbiorów ściennych (`walls/d1`, `walls/d2`). Każda predykcja przeszła przez detekcję kąta obrotu (HoughLinesP) i korektę rotacji przed inferencją.

3. **Filtracja predykcji** — surowe predykcje zostały przefiltrowane algorytmicznie:

- próg ufności: $\text{conf} \geq 0.4$
- maksymalny obszar: $\text{area} < \text{P90}$ rozkładu
- overlap między predykcjami: $\text{IoU} < 0.3$
- deduplikacja drzwi: $\text{pokrycie} < 0.9$
- usunięcie drzwi zachodzących na okna: $\text{IoU} > 0.2 \rightarrow$ usunięcie drzwi

4. **Merge** — połączono oryginalne adnotacje ścian z wyfiltrowanymi pseudo-etykietami, tworząc finalny zbiór `combined_dataset`.

2.4 Docelowy zbiór (combined_dataset)

Split	Obrazy	Ściany	Drzwi	Okna
train	13 563	358 066	99 875	59 914
valid	3 225	85 546	24 479	14 890
test	1 874	52 111	14 066	8 597

Wszystkie etykiety w formacie YOLO bbox (5 pól: `class cx cy w h`). Klasy: **0 = wall**, **1 = door**, **2 = window**.

3. Trening

3.1 final_v1

Trenowany od podstaw (pretrained `yolo11s.pt` → weights COCO).

Parametr	Wartość
architektura	YOLO11s (9.4 M params)
optimizer	AdamW
lr0	0.001667
cos_lr	True
warmup_epochs	3
batch	16

Parametr	Wartość
imgsz	1 024
epochs	50
close_mosaic	15
mosaic	1.0
fliplr	0.5
flipud	0.0
amp	True
patience	25

Czas treningu: ~16 h na NVIDIA RTX 4070 Ti SUPER (16 GB VRAM).

3.2 final_v2

Dotrenowanie final_v1 przez kolejne 50 epoch (fine-tuning z niższym LR).

Parametr	final_v1	final_v2
checkpoint	yolo11s.pt	final_v1/best.pt
lr0	0.001667	0.0005
warmup_epochs	3	0
close_mosaic	15	5

4. Porównanie modeli

4.1 Metodologia

Modele ewaluowano na **niezależnym zbiorze testowym** — oryginalne, niemodyfikowane dane z `walls_doors_windows/d1/test` + `walls_doors_windows/d2/valid` (572 obrazy, czyste adnotacje Roboflow, nigdy nie użyte w pseudo-labelingu). Metryki liczone przez `torchmetrics.detection.MeanAveragePrecision` (IoU threshold 0.5, max detections 100).

4.2 final_v1 vs final_v2

Klasa	Metryka	final_v1	final_v2	Zmiana
wall	mAP50	77.06%	78.19%	+1.13pp
	mAP	41.63%	42.61%	+0.99pp
door	mAP50	79.98%	79.04%	-0.93pp
	mAP	39.53%	39.39%	-0.14pp
window	mAP50	88.13%	88.33%	+0.20pp
	mAP	54.46%	54.83%	+0.37pp

Wniosek: final_v2 daje nieznaczną poprawę na ścianach i oknach kosztem minimalnego regresu na drzwiach. Różnice są w granicach statistical noise. **final_v1 rekomendowany jako domyślny** — w praktyce lepiej radzi sobie z drzwiami, które są najtrudniejszą klasą (małe obiekty, duża zmienność).

4.3 Podejście jednomodelowe vs dwumodelowe

Porównanie final_v1 (1 model, 3 klasy) z podejściem dwumodelowym (**walls_v1** + **doors_windows_v2**) przy imgsz=1024:

Klasa	final_v1	walls_v1	dw_v2
wall mAP50	77.06%	76.62%	N/A
door mAP50	79.98%	N/A	~0%
window mAP50	88.13%	N/A	72.60%

Wniosek: jeden model 3-klasowy bije podejście dwumodelowe na każdej klasie. Model **doors_windows_v2** ma szczególnie słabe wyniki na drzwiach (0.01% mAP50) — przewiduje zbyt duże bounding boxy (~90×100 px zamiast ~80×18 px), ponieważ był trenowany na zbiorze o innych charakterystykach wymiarowych drzwi.

4.4 Wpływ rozdzielczości wejściowej

Porównanie final_v1 przy imgsz=640 vs imgsz=1024:

Klasa	640	1024	Różnica
wall mAP50	72.41%	77.06%	+4.65pp
door mAP50	74.65%	79.98%	+5.33pp

Klasa	640	1024	Różnica
window mAP50	83.28%	88.13%	+4.85pp

Wniosek: wyższa rozdzielczość wejściowa konsekwentnie daje ~5pp wzrostu mAP50. imgsz=1024 jest rekomendowane pomimo wyższych wymagań obliczeniowych.

5. Wytrenowane modele (weights/)

Wszystkie wagi dostępne w katalogu [weights/](#) na GitHub:

Plik	Model	Zbiór treningowy	Wielkość
<code>final_v1.pt</code>	YOLO11s detect, 3 klasy	combined_dataset (z pseudo-labelingiem)	19 MB
<code>final_v2.pt</code>	YOLO11s detect, 3 klasy	fine-tune final_v1	19 MB
<code>raw_3class.pt</code>	YOLO11s detect, 3 klasy	walls_doors_windows d1+d2 (surowe)	19 MB
<code>doors_windows_v2.pt</code>	YOLO11s detect, 2 klasy	merged_doors_windows	19 MB
<code>walls_v1.pt</code>	YOLO11s detect, 1 klasa	walls_only (podejście dwumodelowe)	19 MB
<code>train_full_obb.pt</code>	YOLO11s-OBB, 1 klasa	merged OBB walls	19 MB

6. Skrypty pipeline'u

Skrypt	Opis
<code>train_doors_windows.py</code>	Trening modelu pośredniego (drzwi+okna) na <code>merged_doors_windows</code>
<code>pseudo_label_walls.py</code>	Pseudo-labelowanie drzwi/okien na zbiorach ściennych z detekcją kąta
<code>flatten_pseudo.py</code>	Splaszczanie struktury pseudo-etykiet do jednego katalogu
<code>filter_predictions.py</code>	Algorytmiczna filtracja pseudo-etykiet
<code>merge_final.py</code>	Merge oryginalnych ścian + pseudo-etykiet → <code>combined_dataset</code>
<code>train_final.py</code>	Trening główny (3 klasy) na <code>combined_dataset</code>

Skrypt	Opis
<code>train_final_v2.py</code>	Dotrenowanie final_v1 (fine-tune, niższe LR)

6. Wnioski

1. **YOLO11s z imgsz=1024** na 3-klasowym zbiorze osiąga najlepsze rezultaty (mAP50: wall=77%, door=80%, window=88%).
2. **Pseudo-labeling** skutecznie rozszerzył zbiór o brakujące klasy, umożliwiając trening modelu na danych, które w formie źródłowej nie zawierały kompletnych adnotacji.
3. **Podejście jednomodelowe** (1 model, 3 klasy) znacząco przewyższa podejście dwumodelowe (2 osobne modele dla ścian i drzwi/okien), szczególnie na klasie drzwi.
4. **Fine-tuning** (final_v2) dał marginalną poprawę — dalsze epoki nie przynoszą znaczącego zysku, model osiągnął plateau.
5. **1024px > 640px** — wyższa rozdzielczość konsekwentnie poprawia jakość detekcji o ~5pp mAP50, kosztem ~2× dłuższego czasu inferencji.